传智杯 · Vibe Coding 挑战赛

机房故障智能诊断助手

对话式排查 · AI 驱动 · 面向高校机房运维

作品名称	机房故障智能诊断助手
选题方向	场景化轻量 RAG 知识库
技术栈	Python · Flask · DeepSeek API · 原生前端
当前版本	v0.10.0
代码仓库	github.com/liyishuo1/dsh-fault-diagnosis
作者	李一硕

本文档由项目开发过程实时记录整理而成

目 录

一、项目概述 03

1.1 真实需求来源

1.2 要解决的问题

1.3 作品定位

二、系统架构 04

2.1 整体结构

2.2 模块职责

2.3 数据存储设计

三、核心技术实现 07

3.1 对话式逐层排查

3.2 轻量 RAG 知识库

3.3 知识库自动沉淀

3.4 设备状态四态模型

3.5 并发写入保护

四、意图控制策略 13

4.1 系统提示词设计

4.2 如何约束 AI 的输出

4.3 提示词的迭代过程

五、工程质量保障 17

5.1 自动化测试

5.2 持续集成

5.3 典型缺陷与修复

六、可复现性论证 22

6.1 环境与依赖

6.2 复现步骤

6.3 演进链路

七、AI 协作过程 26

7.1 协作方式

7.2 典型协作案例

一、项目概述

1.1 真实需求来源

本作品的需求直接来自作者本人的日常运维工作，而非虚构场景：

- 负责 **23 间机房教室、约 1840 台计算机设备** 的日常维护
- 管理一支 **50 人的学生运维团队**
- 每周值班 3–4 次，组织每周两次机房大扫除

1.2 要解决的问题

在高频运维工作中，暴露出三个具体痛点：

痛点	具体表现
高频故障重复出现	无法开机、网络中断、外设失灵等问题反复发生，每次都从头排查
排查依赖个人经验	新人上手慢，同一个故障不同人处理方式不一致，处理标准不统一
经验无法沉淀	老管理员毕业后经验随之流失，后来者仍需重新摸索

> **为什么这些痛点值得解决：** 机房运维的核心矛盾是「故障种类有限但处理经验分散在人脑里」。 1840 台设备的故障模式其实高度重复，若能沉淀成可检索的工具， 就能把「依赖个人」变成「依赖系统」。

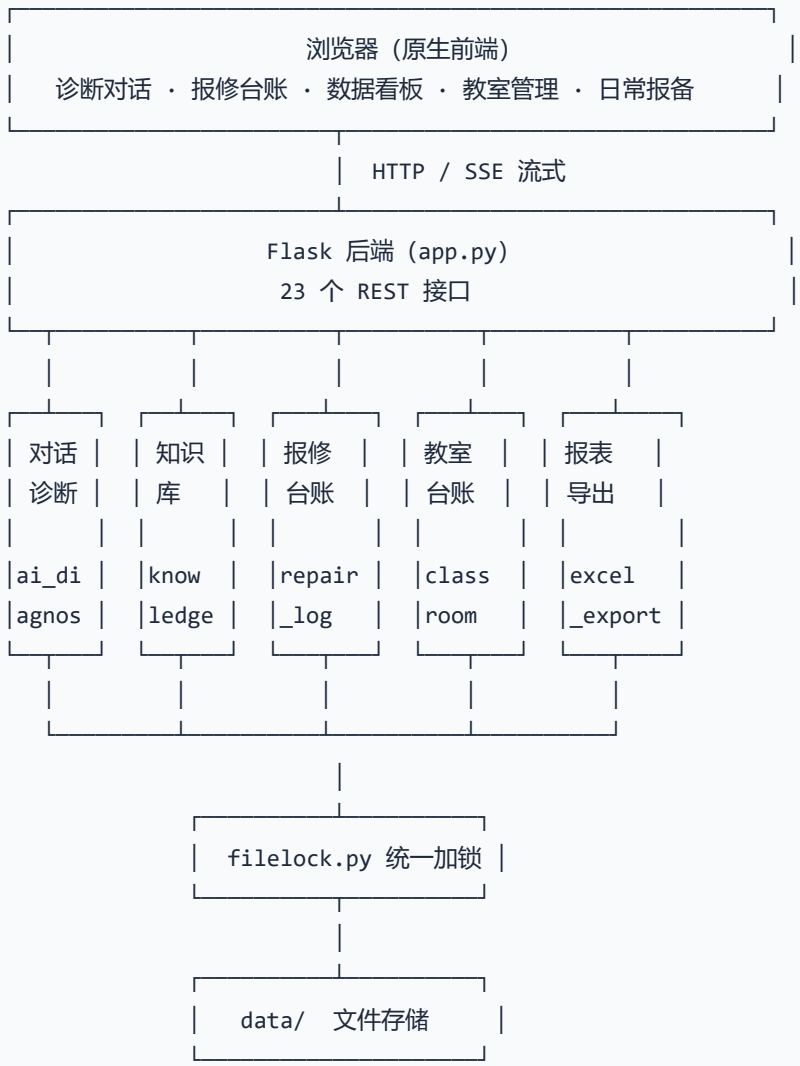
1.3 作品定位

本作品把机房运维经验沉淀成**可检索、可复用的诊断工具**，降低对个人经验的依赖。 与常见的「设备管理系统」不同，本作品的切入点是**故障排查过程本身**：

维度	本作品的做法
输入方式	自然语言描述故障，不需要记忆关键词或分类编号
交互形式	对话式逐层追问，一次只问一个问题，模拟老师傅带徒弟的过程
知识来源	本校机房真实案例库 + AI 通用知识，并通过检索增强（RAG）注入
闭环设计	诊断结论自动写入报修台账，形成「排查 → 记录 → 统计 → 报表」闭环

二、系统架构

2.1 整体结构



2.2 模块职责

模块	行数	职责
`ai_diagnosis.py`	305	对话式诊断；维护会话历史；解析 AI 标记；数量解析
`knowledge.py`	494	知识库检索（轻量 RAG）；分类推断；自动沉淀
`repair_log.py`	430	报修台账 CRUD；状态机；数量统计；CSV 迁移
`classroom.py`	300	教学楼 / 教室两级台账；四态统计
`excel_export.py`	420	面向领导的一页纸报表（单表汇总）
`daily_task.py`	216	日常报备待办清单
`filelock.py`	86	跨平台文件锁（fcntl / msvcrt）
`app.py`	780	Flask 后端；23 个接口；SSE 流式输出

分层原则：业务逻辑全部放在独立模块中，`app.py` 只做 「接收请求 → 调用模块 → 返回 JSON」的转发，不包含业务规则。 这样同一套逻辑既能被 Web 界面调用，也能被命令行 `main.py` 调用。

2.3 数据存储设计

本项目**不使用数据库**，全部采用文件存储。这是刻意的选择：

> **为什么不用数据库：** 本工具部署在机房现场的单机上，数据量极小（几十条记录 / 几个文件）， 引入 SQLite 或 MySQL 会增加部署复杂度，却带不来实际收益。 更重要的是——**纯文件便于人工查看与直接编辑**， 故障现场往往需要管理员直接用记事本打开台账确认， 这是数据库做不到的。

文件	是否提交	用途
`data/fault_cases.json`	✅ 提交	手工整理的知识库（23 条案例）
`data/fault_cases_auto.json`	❌ 忽略	程序自动沉淀的案例（运行时数据）
`data/repair_records.csv`	❌ 忽略	报修台账（含真实教室号，不公开）
`data/classrooms.json`	❌ 忽略	教室与设备台账
`data/daily_tasks.json`	❌ 忽略	日常报备待办

> **知识库分两个文件的设计理由：** 手工整理的经验应当随仓库传递、可评审、可回滚； 而程序自动生成的数据属于运行时产物，不应污染提交记录。 两者分离后，程序只写 `_auto` 文件，永不覆盖手工知识库。

三、核心技术实现

3.1 对话式逐层排查

传统做法是用户输入故障后，一次性输出「5 个可能原因 + 5 个大步骤」， 内容长达上千字。在故障现场，管理员根本看不完，也无法根据实际情况推进。

本作品改为**对话式逐层追问**：

```
[用户] 501 教室电脑打不开
[AI]   按开机键后，机箱指示灯亮吗？风扇转吗？

[用户] 按电源没反应，灯不亮
[AI]   检查一下机箱后面的电源线两头有没有插紧，插紧后再按开机键，有反应吗？

[用户] 换了电源线还是不行
[AI]   看一下机箱背面电源上的开关是不是被关到"O"那边了，拨到"I"再开机试试？

[用户] 开关是开的，灯也不亮
[AI]   把电脑插到旁边正常的机位上试试，能开机吗？

[用户] 换了机位就能开
[AI]   这间教室有几台是这个情况？

[用户] 有 3 台
[AI]   【诊断结束】
       原因：这几个机位的插座没电（供电故障），主机本身没问题。
       数量：3 台
       处理：先确认是不是这几路插座对应的空气开关跳闸了……
       【需要报修】
```

关键实现——维护完整对话历史，让 AI 记住排查过程：

```
class DiagnosisSession:
    def __init__(self):
        self.history = []          # 对话历史
        self.classroom = ''        # 教室号
        self.quantity = 1          # 损坏数量
        self.need_repair = False   # 是否需报修
        self.self_fix = False      # 是否可自修

    def send(self, user_input):
        self.history.append({"role": "user", "content": user_input})
        response = self.client.chat.completions.create(
            model=MODEL,
            messages=self.history,      # 带上完整历史
            temperature=0.3,
            max_tokens=300,             # 限制长度，强制精简
        )
        reply = response.choices[0].message.content.strip()
        self.history.append({"role": "assistant", "content": reply})
        return clean_markers(reply), self.finished, self.need_repair
```

3.2 轻量 RAG 知识库

早期做法：把 `fault_cases.json` 全部案例塞进 system 消息。 案例一多就浪费 token，无关案例还会干扰判断。

现在的做法：先用本地关键词加权检索找出最相关的 Top-3 案例，只注入这几条。

检索算法

权重	命中位置
×3.0	故障分类一致
×2.0	现象描述命中
×0.5	可能原因命中（权重压低，见下）
额外加成	命中关键特征词表（如"蓝屏""断网"给 3.0）

> **为什么「原因」权重只有 0.5：** 原因是排查要点，用词宽泛（如"交换机断电或故障"里的"断电"）， 容易与无关领域偶然重合。曾经出现过查「电脑自动关机」时， 因共享「断电」一词而误命中网络类案例的假阳性。 压低权重并增加相关性判断后解决。

同义词归一

用户说「整间教室断网」，知识库里写的是「无法上网」—— 两边指的是同一件事，但字面完全没有重合，检索得分是 0。

```
SYNONYMS = {
    '上网': ['断网', '上不了网', '无法上网', '不能上网', '连不上网'],
    '开机': ['开不了机', '打不开', '无法开机', '点不亮'],
    '黑屏': ['无显示', '没显示', '屏幕黑', '不显示', '没画面'],
    # ... 共 17 组
}

# 反向索引：说法 → 标准词（模块加载时构建一次）
_SYNONYM_LOOKUP = {}
for _std, _variants in SYNONYMS.items():
    _SYNONYM_LOOKUP[_std] = _std
    for _v in _variants:
        _SYNONYM_LOOKUP[_v] = _std
```

关键在于**必须在切分之前扫描原文**：用户说「教室上不了网」，按标点切分只会得到 `教室上`、`室上不` 这类碎片，`上不了网` 根本不会被提取出来。

粒度一致性

另一个被实际使用暴露的问题：查询「空调漏水」（4 字短句）整段保留，而案例症状是长句、被切成 3-gram（`空调漏`、`调漏水`）—— 两边粒度不一致，交集为空，导致**刚沉淀的新案例用关键词反而搜不到。**

修复：长短句统一产出 3-gram，匹配时放宽为互为子串即算命中。

3.3 知识库自动沉淀

诊断结束后，若结论为「自己修」（已报修的不沉淀，因为结论尚未验证），系统把现象与结论提取成结构化案例写回知识库，实现**越用越聪明。**

```
def save_case(symptom, conclusion, excluded=None, classroom=''):
    # 1) 去重：现象高度相似的不重复添加
    existing = search(symptom, top_n=1, min_score=8.0)
    if existing:
        return False, f'已有高度相似案例 {existing[0].get("id")}'

    # 2) 从结论抽取「原因」和「处理」
    reason = re.search(r'原因[:： ]\s*(.+)', conclusion)
    action = re.search(r'处理[:： ]\s*(.+)', conclusion)

    # 3) 推断分类、生成编号（在合并后的全量案例中避让）
    category = _infer_category(symptom + ' ' + conclusion)
    new_id = f'{prefix}-{max(nums) + 1:03d}'

    # 4) 只追加到自动沉淀文件，不碰手工文件
    _append_auto_case(new_case)
    return True, new_id
```

分类推断的规则设计

分类规则按「越具体越靠前」排列，第一个命中的生效。 这里踩过一个坑：早期版本把「声音」作为音频类的关键词， 导致**「风扇声音特别大」被判成音频故障**（散热问题被当成音响问题）。

```
_CATEGORY_RULES = [
    # 外设类先于音频：'音响''耳机'本身就带"响/机"
    (['鼠标', '键盘', 'U盘', 'usb', '摄像头'], '外设故障'),
    (['投影', '投影仪'], '投影故障'),
    (['打印机', '卡纸'], '打印故障'),
    (['上网', '断网', '网络', '网线', '交换机'], '网络故障'),
    (['蓝屏', '死机', '卡顿', '系统', '重装', '病毒'], '系统故障'),
    (['显示', '黑屏', '花屏', '无信号', '显示器'], '显示故障'),
    # 开机类：补齐 关机/断电/重启/硬件部件
    (['开机', '打不开', '不通电', '关机', '断电', '重启',
      '电源', '主板', '内存', '硬盘', '显卡', '风扇', '散热'], '开机故障'),
    # 音频类放最后，且用具体词
    (['音响', '音箱', '耳机', '话筒', '麦克风', '没声音', '杂音'], '音频故障'),
]
```

3.4 设备状态四态模型

设备的可用状态设计为四种（面向运维）与两种（面向领导报表）两套口径：

系统状态	含义	报表归类
未报修	硬件故障，需要报修	损坏
可自修	非硬件问题，可自行修复	损坏
已报修	已联系维修，等待处理	损坏
已解决	已完结	正常

> **「损坏」的含义要有意说清：** 它指的是**设备能不能用**，与报没报修无关。 因此 损坏 = 未报修 + 可自修 + 已报修。
>
> 界面上最初把这一项标成了「需报修」，但已报修的那几台并不需要**再**报修， 名称与实际含义不符，后来统一改为「损坏」。 这个概念区分很重要：**「坏没坏」是设备属性，「报没报修」是处理进度**， 两者是不同的维度，混在一个标签里就会说不清。

> **为什么报表要合并成两种：** 这份报表是给领导看的，领导不关心修没修、谁去修， 只关心「现在有多少台能用」。因此报表口径为： **正常 + 损坏 = 设备总数**，账目能直接对上。

「不能用台数」的统计口径

一条报修记录可以代表多台设备（如「第三排 5 台都开不了机」），因此统计台数时按**数量字段累加**，而不是数记录条数。

```
def count_machines(records, statuses=None):
    """统计一批记录代表的机器台数（累加数量字段）"""
    total = 0
    for r in records:
        if statuses is not None and r.get('状态') not in statuses:
            continue
        total += get_quantity(r)
    return total
```

3.5 并发写入保护

Flask 默认多线程，多个请求可能同时「读-改-写」同一个数据文件。没有锁时，后写的会覆盖先写的，导致**数据丢失**。

两层锁的设计

最终的实现用了两层锁，这是被两个真实缺陷逼出来的结构：

```
@contextlib.contextmanager
def file_lock(target_path):
    """
    第一层：进程内可重入锁（threading.RLock）
    第二层：跨进程文件锁（flock / msvcrt）
    """
    key = os.path.abspath(target_path)
    local = _get_local_lock(target_path)

    with local:
        # 本线程是否已持有该文件的系统锁 → 本次是内层重入
        if key in _held_paths():
            yield
            return          # 内层不碰系统锁，直接执行

        lock_path = target_path + '.lock'
        fh = open(lock_path, 'w')
        try:
            import fcntl                        # Linux / macOS
            fcntl.flock(fh.fileno(), fcntl.LOCK_EX)
        except ImportError:
            import msvcrt                       # Windows
            msvcrt.locking(fh.fileno(), msvcrt.LK_LOCK, 1)

        _held_paths().add(key)
        yield
        # ……释放锁并清理
```

关键点一：必须加锁之后再读取

```
# 正确做法
with file_lock(AUTO_FILE):
    data = _load_auto()          # 先加锁，再读取
    data['cases'].append(case)
    _write_auto(data)
```

如果先读再加锁，读到的仍是过期内容，锁就白加了。

关键点二：「检查-写入」必须整体持锁

这是开发后期才暴露的一个缺陷。原来「文件不存在则创建」是这么写的：

```
def init_file():
    if not os.path.exists(RECORD_FILE):          # ← 检查
        with open(RECORD_FILE, 'w', ...) as f:   # ← 'w' 是截断写
            csv.writer(f).writerow(FIELDS)
```

多线程同时首次调用时，都会通过 `exists()` 检查—— 因为此刻文件确实还不存在。随后**后到的那个以 `'w'` 模式打开， 把先到者刚写入的内容全部清空。**

> **这个缺陷只在 Linux 上暴露，而且时好时坏。** Windows 下 `msvcrt` 加锁与线程调度更慢， 恰好错开了竞态窗口，因此本地测试一直通过， 直到 CI 在 Ubuntu 上跑才暴露出来。
>
> 实测复现：60 轮并发首次创建，**47 轮丢数据。** 修复后同样的压测 0 轮失败。

修复方式有两处：一是让 `init_file` 全程持锁， 创建时改用 `'x'` 模式做二次保护（并发进入者会拿到 `FileExistsError` 而不是把文件截断）； 二是让 `file_lock` 支持重入。

关键点三：锁必须可重入

第二条修复有一个前提——如果锁不可重入，给 `init_file` 加锁会立刻造成死锁：

```
update_status()
  └ 持有锁
      └ load_records()
          └ init_file()
              └ 又要加锁    ← 锁不可重入则死锁
```

业务代码里这种「持锁后调用另一个也会加锁的函数」的写法是合理且常见的， 因此锁在这一层做成可重入：同一线程重复进入时只累加计数， **只有最外层才真正去拿系统锁。**

不能直接依赖 `flock` 的重入行为——同一进程内对同一文件 用两个不同 fd 加锁，各平台表现并不一致。所以重入由 Python 层 自己的 `RLock` 负责，系统锁只在最外层拿一次。

> **附带收益：**修复后测试耗时从约 32 秒降到约 6 秒。 说明此前线程在锁上存在大量无谓等待。

验证方式：50 轮 × 12 线程高并发压测，0 轮失败； 另有两项防死锁测试（4 层嵌套加锁、锁内调用 `load_records`）， 均已固化进 `tests/test_concurrency.py`。

四、意图控制策略

4.1 系统提示词设计

系统提示词是本项目最核心的设计。它的目标不是「让 AI 更聪明」， 而是**约束 AI 的行为边界**——把通用大模型改造成一个守规矩的运维助手。

```
你是一名高校机房的运维工程师，正在通过对话帮学生管理员排查设备故障。

## 核心规则（必须严格遵守）

1. 每次回复只做一件事：要么问一个问题，要么给出结论。
   绝不一次列出多个问题或多个步骤。

2. 教室号已知时绝不再问。只有完全不知道教室时才问。

3. 第一步问清具体表现。如果用户描述很模糊（如"电脑打不开"），
   先问清楚具体现象，不要跳到排查。

4. 提问要多样化，禁止套模板。
   不要每次都问"指示灯亮吗？风扇转吗？"
   根据故障类型选择不同的切入角度。

5. 根据回答推进：用户说"不行"就排除这个原因，进入下一个全新的可能。
   绝不重复问同一个问题。

6. 用户答非所问时：先简短确认他的信息，再把原来的问题换个更简单的说法。

7. 确定原因后，先问损坏数量。
   拿到数量后再输出结论。

8. 数量要记住，不要写错。绝不能写 0 台。

## 严禁
- 严禁一次列出多个排查步骤
- 严禁输出大段解释和原理
- 严禁重复已经排除的检查项
- 严禁使用「首先、其次、最后」这种结构化列点
```

4.2 如何约束 AI 的输出

除了提示词，还有三道工程约束：

手段	作用
`max_tokens=300`	物理限制回复长度，强制精简。写得长会被直接截断
`temperature=0.3`	降低随机性，让排查步骤稳定、专业，不出现发散内容
结构化标记	用 `【诊断结束】` `【需要报修】` `【可自修】` 三个标记，让程序能可靠地解析 AI 意图，而不必猜自然语言

标记解析与状态映射

```
def apply_markers(session, reply):
    """根据 AI 回复里的标记，更新会话状态"""
    if REPAIR_MARKER in reply:
        session.need_repair = True
    if SELF_FIX_MARKER in reply:
        session.self_fix = True
    if DONE_MARKER in reply:
        session.finished = True
        session.quantity = parse_quantity(reply)   # 从结论解析数量
    return reply


def parse_quantity(text):
    """容错解析：支持 "3 台" / "三台" / "十五台" """
    m = re.search(r'数量[：: ]\s*(\d+)', text)
    if m:
        return max(int(m.group(1)), 1)
    # 中文数字、兜底匹配……
    return 1      # 解析不出时保守记为 1，绝不能是 0
```

4.3 提示词的迭代过程

提示词不是一次写对的，而是**根据实际对话暴露的问题逐步补充规则**：

发现的问题	补充的规则
用户说「电脑打不开」（模糊描述）时，AI 直接跳到排查电源开关	新增第 3 条：**第一步永远是问清具体表现**
用户答非所问时，AI 把上一个问题换个说法又问一遍	新增第 6 条：**先确认信息，再用更简单的方式重问**
AI 每次都问「指示灯亮吗？风扇转吗？」，像套模板	新增第 4 条：**提问要多样化，按故障类型选择切入角度**
用户已说明数量，AI 结论里仍写「数量：1 台」	新增第 8 条：**数量要记住，不能写错，绝不能是 0**

> **经验：提示词需要根据实际测试结果迭代。** 第一版写出来的提示词往往不完善，必须通过真实对话发现问题、 再针对性补充规则。这条迭代记录已保存在 `docs/开发过程记录.md` 中。

五、工程质量保障

5.1 自动化测试

项目包含 **159 个自动化测试**，覆盖业务逻辑的关键路径与历史缺陷：

测试文件	用例数	覆盖内容
`test_knowledge.py`	37	检索质量、假阳性防护、自动沉淀、手工/自动数据隔离
`test_repair_log.py`	12	台账 CRUD、状态排序、旧 CSV 表头迁移
`test_classroom_daily.py`	30	教室台账校验、四态统计、待办排序与逾期判定
`test_quantity_and_status.py`	38	数量解析、状态判定与迁移、批量登记不丢数据
`test_concurrency.py`	10	并发写入不丢数据、文件锁正确性、锁可重入防死锁
`test_excel_export.py`	32	报表口径、纵向合并、原因精简、合计行

> **测试设计原则：只测不变的契约，不测易变的实现。** 例如测试「正常 + 损坏 = 设备总数」这条业务规则， 而不是测试某个函数的具体实现细节——前者改代码时仍成立， 后者稍作重构就会误报。

5.2 持续集成

```yaml
name: tests
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        # Windows 也要测：文件锁在 Windows 走 msvcrt、Linux 走 fcntl，
        # 两条分支都必须验证
        os: [ubuntu-latest, windows-latest]
        python-version: ['3.10', '3.12']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install -r requirements-dev.txt
      - run: python -m pytest tests -v
```

每次推送到 main 或提交 PR，自动在 **4 种环境组合**上跑一遍测试。

5.3 典型缺陷与修复

以下是开发过程中真实发现并修复的五个典型缺陷，均有测试保护：

缺陷 1：异常被静默吞掉，知识库悄悄失效

现象：没有任何报错，程序运行正常，但知识库**从未被成功加载。**

```
# 有问题的代码
try:
    with open(data_file, 'r', encoding='utf-8') as f:
        cases = json.load(f)['cases']
except Exception:
    return ""          # ← 所有异常被吞掉

# 调用方
if ctx:                # 空字符串判定为假 → 静默跳过知识库
    ...
```

根因： `fault_cases.json` 有两处逗号缺失导致 JSON 解析失败， 但异常被 `except` 吞掉并返回空字符串，程序照常运行，表面上完全看不出问题。

修复：补全逗号；并加上实际验证——调用函数确认返回值非空。

> **经验：`except` 后返回空值是最危险的失败模式。** 它把「配置坏了」伪装成「没有配置」。崩溃会立刻暴露问题， 而静默失败只会让功能「看起来能用、实际少了一块」。

缺陷 2：检索同义词缺口

现象：用户搜「整间教室断网」→ 无匹配。但知识库里明明有网络案例。

根因：`断网` 是关键词，但只对「用户说断网」加分； 案例正文写的是「无法上网」——这个加分永远加不上， 同义表达之间没有打通。

修复：新增同义词归一表，并在切分前扫描原文。

缺陷 3：并发写入丢数据

现象：两个诊断同时结束时，台账只留下一条记录。

根因：「读-改-写」无锁，后写的覆盖先写的。

修复：引入跨平台文件锁，并确保**加锁之后再读取**。

缺陷 4：图表颜色与主题冲突

现象：下拉选项在深色主题下是白底浅字，几乎无法阅读。

根因：只给 `<select>` 控件设了深色样式， 没给下拉出来的 `<option>` 设。

修复：新增全局 `select option` 规则统一处理。

缺陷 5：文件初始化的「检查-写入」竞态

现象：CI 里 Ubuntu 两个 Python 版本时好时坏地失败， 而 Windows 始终通过。失败只用了 15~19 秒，像是并发用例挂掉。

根因：数据文件「不存在则创建」这一段的检查与写入之间没有锁， 而写入用的是 `'w'` 模式——它是**截断写**。 多线程同时首次调用时，后到的会把先到者刚写的内容清空。

```
if not os.path.exists(RECORD_FILE):          # 多个线程都能通过
    with open(RECORD_FILE, 'w', ...) as f:   # 'w' 截断，覆盖先写入的
```

> **为什么本地一直没发现：** Windows 的 `msvcrt` 加锁与线程调度更慢，恰好错开了竞态窗口。 这个缺陷只在 Linux 上暴露，而且不是每次都触发—— 很容易被误判为「测试不稳定」而加个重试糊过去。 但它会真的丢数据：两个管理员同时提交报修，就可能少一条记录。
>
> 实测复现：60 轮并发首次创建，**47 轮丢数据**。

修复：整个检查-创建过程持锁；创建改用 `'x'` 模式 （并发进入者会拿到 `FileExistsError`，而不是把文件截断）。 由于这一步加锁后与已有调用路径形成嵌套，同时把 `file_lock` 改造成可重入（详见 3.5 节）。

验证：修复后 60 轮压测 0 失败；新增回归测试并确认 **移除修复后该测试会立即失败**，确保测试真正覆盖了这个缺陷。

六、可复现性论证

6.1 环境与依赖

项目	要求
操作系统	Windows / Linux / macOS
Python	3.10 或更高（开发环境 3.15.0）
依赖库	仅 3 个：`openai` `flask` `openpyxl`
数据库	无需数据库
前端构建	无需构建工具（原生 HTML/CSS/JS）
API Key	DeepSeek API Key（从环境变量读取）

6.2 复现步骤

```
# 1）克隆仓库
git clone git@github.com:liyishuo1/dsh-fault-diagnosis.git
cd dsh-fault-diagnosis

# 2）安装依赖（仅 3 个库）
pip install -r requirements.txt

# 3）配置 API Key（不写入代码，从环境变量读取）
#    Windows PowerShell：
[Environment]::SetEnvironmentVariable("DEEPSEEK_API_KEY", "你的key", "User")
#    设置后需要新开一个终端

# 4）启动
cd src
python app.py
# 浏览器访问 http://127.0.0.1:5000

# 5）运行测试（可选，159 个用例）
pip install -r requirements-dev.txt
python -m pytest tests -v
```

> **复现时间：约 5 分钟。** 无需数据库、无需前端构建、无需 Docker。 这也是刻意保持「零额外依赖」的原因——降低复现门槛本身就是可复现性的一部分。

6.3 演进链路

代码仓库保留了完整的细粒度提交历史，共 **55 个提交**， 首次提交仅包含 `.gitignore`（不是一次性 initial commit）。 每次提交对应一个功能模块或一次修复：

阶段	版本	关键提交
起步	v0.1.0	初始化项目结构 → 添加故障知识库 → 实现本地查询
	v0.2.0	决策树逐步诊断（**验证后暂缓**，维护成本过高）
AI 接入	v0.3.0	接入大模型 API → 主程序改为 AI 优先诊断
	v0.4.0	改为对话式逐层排查
	v0.5.0	Web 界面 → 流式输出 → 数据看板
业务闭环	v0.6.0	教室设备台账 → 报修闭环 → 日常报备
	v0.7.0	知识库检索增强（轻量 RAG）→ 自动沉淀
	v0.8.0	数据分离 → 检索粒度修复 → 并发保护
工程化	v0.9.0	159 个测试 → CI → 统一文件锁 → 会话回收
	v0.10.0	Excel 报表 → 四态模型 → 界面重构

> **关于「决策树方案暂缓」：** 这一版是**做出来、验证过、再决定放弃**的，而非凭空判断。 实测发现每增加一种故障都要手工编写大量节点，且分支写死、 无法覆盖真实故障的多样性。代码保留在仓库中备查。这条记录本身就是「方案不是一次做对的」的证明。

七、AI 协作过程

7.1 协作方式

本项目全程使用 AI 辅助开发。协作方式不是「让 AI 写完整代码」， 而是**人负责判断与验证，AI 负责实现与建议**：

环节	人的职责	AI 的职责
需求分析	提出真实痛点，定义验收标准	给出数据结构与实现方案
编码实现	审查代码逻辑，要求逐段解释	生成代码，说明设计取舍
问题排查	**提供完整报错信息与复现步骤**	定位根因，给出修复方案
效果验证	**实际运行验证，发现与预期不符之处**	根据反馈调整

7.2 典型协作案例

案例：从「6 张表」到「1 张表」的需求纠偏

第一轮 Prompt（我）：

> 设计导出功能：将报修记录导出成 `excel`。
> 表格顺序按照未报修-已报修-已解决处理，按照教学楼-教室进行分类。
> 要求体现教室历史故障原因和次数，并且反应现状：
> 目前教室有多少台能用，多少台不能用，不能用的一一对应好损坏时间和原因

AI 实现：生成了 6 个工作表（总览、故障总表、教室现状、损坏明细、故障统计、未录入台账）。 **第二轮反馈（我）：**

> 导出的格式完全不行，你听我的要求，说完全部需求后再去做

第三轮（我给出完整需求）：

> 用途需要领导看，要直观看出每间教室的情况。
> 你将工作表说明全部删除，表的最左侧是教学楼，
> 一次向下排，教学楼的右边接着对应的教室。
> 表头最上面是教学楼-教室-总设备数量-正常运行设备数量-损坏数量及其原因

进一步澄清（我）：

> 因为是领导看，领导不管修没修，他只管现在能运行多少，
> 所以待修理和未报修都算作损坏，已解决不用单拎出来，
> 放入正常运作数量里。只留这一张。

最终结果：单张汇总表，一间教室一行，「正常 + 损坏 = 设备总数」，账目可直接对上。

> **这个案例说明什么：** AI 的实现能力很强，但**它不掌握业务的真实约束**—— 它不知道报表是给谁看的、领导关心什么。 第一版 6 张表在技术上没有错误，但解决的不是真问题。 把「用途、受众、判断标准」讲清楚，比反复描述格式更有效。

案例：AI 主动发现的隐藏缺陷

在实现「自动沉淀数据分离」时，AI 在回归测试中发现了一个更严重的问题：

> 新沉淀一条"机房空调漏水导致地板大面积湿滑"
> 然后搜"空调漏水" → 无匹配 ❌

刚存进去的案例，用关键词居然搜不到——等于沉淀了也没用。 根因是切分粒度不一致（查询是 4 字整段，案例被切成 3-gram）。

同一次测试还暴露出 AI 先前加的一个「降噪过滤器」是错的： 它只保留「能对应到已知词表」的词， 导致**词表外的新领域（如"空调漏水"）永远无法沉淀。** 该过滤器已被移除，改在打分环节解决假阳性。

> **AI 也会犯错，而且可能错得很隐蔽。** 那个过滤器初衷是好的（挡掉跨词碎片噪音）， 但实现方式等于给知识库设了一道隐形准入限制。 **凡是「让系统只能认识预先知道的东西」的设计， 都会在遇到真实世界的新情况时失效。** 这个教训只有靠真实测试才能发现，靠阅读代码是看不出来的。

7.3 完整对话记录

项目保留了与 AI 协作的**完整原始对话记录**（未经改写）：

文件	内容
`docs/AI对话记录-完整版.md`	全部 11 轮对话，含每轮的 Prompt、工具调用与 AI 回复
`docs/AI对话记录-关键片段.md`	精选 6 段「提问 → 方案 → 踩坑 → 修复」的完整过程

这两份文件由 `tools/export_conversation.py` 从 AI 工具的会话日志 **自动导出**，可保证过程真实可追溯，不存在事后编撰的可能。

> **为什么这份材料重要：** 评审要求「过程可复现」，而最容易造假的恰恰是「开发过程叙述」。 从工具日志直接导出，是唯一能自证真实的方式。

李一硕 · 2026 年 10 月